

2018 年 863 参考答案

一、选择题

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
B	C	D	B	B	D	C	D	B	D	B	C	D	B	D

二、判断题

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
×	×	×	√	×	×	×	√	×	×	×	×	×	×	×

三、填空题

- 深度优先搜索
- 队列
- $O(e)$ 稀疏
- 出度 入度
- $O(n^2)$
- $2^{(i-1)}$
- $n+1$
- 冒泡排序 快速排序
- 中序遍历的直接前驱 中序遍历的直接后继
- 线性结构 非线性结构

四、简答题

先序: AEFBGCDHIJKT
中序: EFAGBCHKIJTD

1. 后序遍历序列: FEHGKJ IHD CBA

2. (1) 邻接矩阵

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	0	1	0	0
C	1	0	0	1	0	0
D	0	1	1	0	1	0
E	0	0	0	1	0	1
F	0	0	0	0	1	0

(2) 深度优先搜索: A B D E F C (不唯一)
广度优先搜索: A B C D E F

3. (1) 有向图带权邻接矩阵

	V_0	V_1	V_2	V_3	V_4	V_5
V_0	∞	30	30	20	∞	60
V_1	∞	∞	25	∞	∞	∞
V_2	∞	∞	∞	∞	∞	35
V_3	∞	∞	∞	∞	∞	50
V_4	∞	∞	∞	∞	∞	∞
V_5	∞	∞	∞	∞	45	∞

(2) 迪杰斯特拉:

终点	从 V_0 到各终点的 dist 的值和最短路线				
V_1	$V_0 \rightarrow V_1: 30$	$V_0 \rightarrow V_1: 30$			
V_2	$V_0 \rightarrow V_2: 30$	$V_0 \rightarrow V_2: 30$			
V_3	$V_0 \rightarrow V_3: 20$				
V_4	∞	∞	∞	∞	105 $V_0 \rightarrow V_5 \rightarrow V_4$
V_5	$V_0 \rightarrow V_5: 60$	$V_0 \rightarrow V_5: 60$	$V_0 \rightarrow V_5: 60$	$V_0 \rightarrow V_5: 60$	
V_T	$\{V_0, V_3\}$	$\{V_0, V_3, V_1\}$	$\{V_0, V_3, V_1, V_2\}$	$\{V_0, V_3, V_1, V_2, V_5\}$	$\{V_0, V_3, V_1, V_2, V_5, V_4\}$
	V_3	V_1	V_2	V_5	V_4

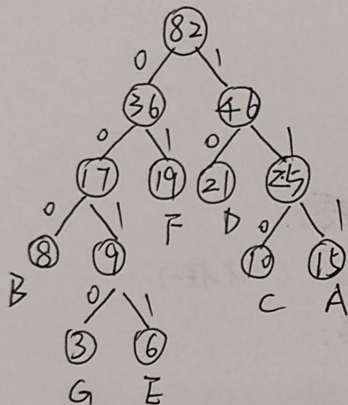
4. (1) ①将这 n 个结点, 分别作为 n 棵仅含一个结点的二叉树, 构成森林 F

②构造一棵新结点, 从 F 中选取两棵根结点权值最小的树
作为新结点的左右子树, 并且将新结点的权值置为左右
子树根结点的权值之和.

③从 F 中删除刚选出的两棵树, 同时将新的树加入到 F 中.

(2) ④重复②和③, 直至 F 中剩下一棵树 (叙述内容不唯一)

3, 6, 8, 10, 15, 19, 21



A: 111

B: 000

C: 110

D: 10

E: 0011

F: 01

G: 0010.

(3)

优点:

①平均码长是不会改变的, 编码效率唯一性

②哈夫曼编码是依据字符出现的频率来构造的, 比较精准.

5. a) typedef struct node

```
{ int key;
  datatype data;
```

```
  struct node *next;
```

```
} HashTable;
```

或 #define MAXSIZE.

```
int hash[MAXSIZE];
```

(2)

0	1	2	3	4	5	6	7	8	9	10	11
55	11	45	33	26	27	27	18			21	

$H(55) = 0$

$H(21) = 10$

$H(18) = 7$

$H(27) = 5$

$H(11) = 0, H_1(11) = 1$

$H(45) = 1, H_1(45) = 2$

$H(33) = 0, H_1(33) = 1, H_2(33) = 2, H_3(33) = 3$

$H(26) = 4$

$H(27) = 5, H_1(27) = 6$

$$(3) \text{ ASL} = \frac{1+1+1+1+2+2+4+1+2}{9} = \frac{5}{3}$$

五、设计题

1. ① 从键盘可以读入顶点个数, 顶点信息, 以及边的信息.

```
void CreateGraph (MGraph *g)
```

```
{
    int i, j, k;
    int n, e; // n表示顶点个数, e表示边的个数
    printf("请输入顶点的个数:");
    scanf("%d", &n);
    printf("\n");
    printf("请输入边的个数:");
    scanf("%d", &e);
```

```
    for (int i=0; i<n; i++)
```

```
        scanf("%d", &g->ver[i]); // 初始顶点
```

```
    for (i=0; i<n; i++)
```

```
        for (j=0; j<n; j++)
```

```
            g->AdjMatrix[i][j]=0;
```

```
    for (k=0; k<e; k++)
```

```
{
    printf("输入边 (i, j):");
```

```
    scanf("%d %d", &i, &j);
```

```
    g->AdjMatrix[i][j]=1;
```

```
    g->AdjMatrix[j][i]=1;
```

```
}
```

```
    printf("\n");
```

```
}
```

(2). 广度优先算法.

```
void BFS (MGraph *g)
```

```
{
    int Q[MAXSIZE], front, rear, visited[MAXSIZE];
```

```
    front = rear = -1;
```

```
    int k;
```

```
    for (i=0; i<n; i++)
```

```
        visited[i]=0;
```

```
    for (i=0; i<n; i++)
```

```
        if (!visited[i])
```

```
            {
                printf("%d", g->ver[i]);
```

```
                visited[i]=1;
```

```
                Q[++rear]=i;
```

```
            }
            while (front != rear)
```

```
            {
                k = Q[++front];
```

```
                for (int j=0; j<n; j++)
```

```
                    if (g->AdjMatrix[k][j]==1
```

```
                        && !visited[j])
```

```
                        {
                            printf("%d", g->ver[j]);
```

```
                            visited[j]=1;
```

```
                            Q[++rear]=j;
                        }
```

持续判断

所选

7, 2, 0

② 广度优先搜索算法

void BFS (Mgraph G)

{ Queue Q;

for (int i=0; i<n; i++)

visited[i] = FALSE;

InitQueue (&Q);

for (int i=0; i<n; i++) {

if (!visited[i]) {

visited[i] = TRUE;

printf ("%d", G.vexs[i]);

EnQueue (&Q, i);

while (!QueueEmpty(Q)) {

DeQueue (&Q, &i);

for (int j=0; j<n; j++)

if (G.AdjMatrix[i][j] == 1 && !visited[j])

{

visited[j] = true;

printf ("%d", G.vexs[j]);

EnQueue (&Q, j);

}

}

}

}

}

2. 将新元素 x 插入到带头结点的单链表中的元素 $elem$ 之前, 若不存在元素 $elem$, 则插入

void Insert (NodeType *head, dataType x, dataType elem)

{ NodeType *p=head, *s;

while (p->next != NULL && p->next->data != elem) p=p->next;

s = (NodeType *) malloc (sizeof (NodeType));

s->data = x;

s->next = NULL;

if (p->next == NULL) p->next = s;

else { s->next = p->next->next; p->next->next = s; }

}

3. (1) 冒泡排序.

void maopao (SqList &L)

```
{
    int flag; // 变量flag用来本趟排序是否发生了变化
    for (int i=1; i<L.length; i++)
    {
        flag=0;
        for (int j=1; j<=n-i; j++)
        {
            if (L.r[j].key > L.r[j+1].key)
            {
                flag=1;
                L.r[0]=L.r[j];
                L.r[j]=L.r[j+1];
                L.r[j+1]=L.r[0];
            }
        }
        if (flag==0) break;
    }
}
```

2) 折半查找

int zheban (SqList &L, keyType k)

```
{
    int low=1, high=L.length, mid;
    while (low <= high)
    {
        mid=(low+high)/2;
        if (L.r[mid].key == k) return mid;
        else if (L.r[mid].key > k) high=mid-1;
        else low=mid+1;
    }
    return -1;
}
```